

issues begin. It is here a decision about whether work is going to be accepted will occur |
Stage::Design	It is here a gameplan for the issue will be created
Stage::Planning	It is here that implementation timelines will be determined
Stage::Development	It is here work will be done to implement it
Stage::Validation	It is here that requester validation will occur
Stage::Completed	This indicates the issue is completed
Stage::Backlogged	This indicates an issue is backlogged.
Stage::Blocked	This indicates something is blocking moving forward on an issue

System labels

Label	What it means
Zendesk	The issue relates to Zendesk (any instance)
Zapier	The issue relates to Zapier
Calendly	The issue relates to Calendly
Salesforce	The issue relates to Salesforce
Workato	The issue relates to Workato

Types of work labels

Label	What it means
bug	The issue is concerning a bug
feature	The issue is concerning a feature request (or change)
incident	The issue is concerning an incident
administrative	The issue is concerning administrative work

Division labels

These are scoped labels, meaning only one of them can ever be present on an issue at a time. This is because a requester is only going to be stemming from one team.

Label	What it means
Customer::Security	The request is from the Security team
Customer::Engineering	The request is from the Engineering team
Customer::Finance	The request is from the Finance team
Customer::People	The request is from the People team
Customer::Support	The request is from the Support team
Customer::Field	The request is from the Field team
Customer::Marketing	The request is from the Marketing team

Priority labels

Label	What it means
p1	It is a priority 1 (urgent) level issue
p2	It is a priority 2 (high) level issue
p3	It is a priority 3 (medium) level issue

| p4 | It is a priority 4 (low) level issue |

Support roadmap labels

| Label | What it means |

|-----|-----|

| roadmapitem | The issue is pertaining to an item on thhe Support roadmap |

SECTION: security/customer-support-operations/docs/gitlab/personal-data-requests.md

{{% pageinfo color="warning" %}}

This is an information page for the various components of personal data requests we manage.

If you are looking for information about working the Operations components of personal data requests, please see Working Personal Data Requests.

If you are looking for information about managing the components described on this page, please see workflows.

{{% /pageinfo %}}

Form

^{Introduced via support-team-meta1329}

The Personal Data Request form (formerly Account Deletion form) is a simple HTML form generated via GitLab Pages that is used for personal data requests.

The source code for it is located here.

Form submission processor

^{Introduced via support-team-meta1329}

When the form is submitted, the Account Deletion Processor is what processes those request.

Issue processor

^{Introduced via support-team-meta1329}

This runs every hour to process the issues in the tracker.

It handles the assignment of issues to those who will work them and the tagging of the issues.

Triage policies

^{Introduced via support-team-meta1329}

Utilizing the GitLab Triage Gem, the triage policies are a group of conditions and actions that are enacted upon issues within the Account Deletion and Other Requests project.

The source code for it is located here

SECTION: security/customer-support-operations/docs/gitlab/support-super-form.md

^{Introduced via support-team-meta5654}

{{% pageinfo color="warning" %}}

This is an information page for the Support Super form.

If you are looking for information about managing it, please see workflows.

{{% /pageinfo %}}

This form enables the submissions for many types of requests. For information on how each of the requests, please see Support Super Form Processor

How it works

It utilizes GitLab Pages to render the various files into a form. Form submissions from here then trigger a pipeline in the Support Super Form Processor project.

The source project is mirrored to the target project to utilize the permission setup of gitlab-com.

The end result is the primary URL that should be utilized: <https://support-super-form-gitlab-com-support-support-op-651f22e90ce6d7.gitlab.io/>

SECTION: security/customer-support-operations/docs/gitlab/support-team-yaml-files.md

^{Introduced via support-team-meta2159}

{{% pageinfo color="warning" %}}

This is an information page for the Support team YAML files.

If you are looking for information about managing them, please see workflows.

{{% /pageinfo %}}

What are the Support team YAML files

The Support team YAML files are a group of YAML files that contain various data we utilize in

other components at GitLab.

Where are they stored

They are stored within the Support Team project.

How are they used

The data within is compiled into an array of hashes that is then stored in Redis. This is done via the Support Team Redis Sync project.

Anatomy of a YAML file

name

The support team member's name

email

```
{{% alert title="Note" color="warning" %}}  
This should never be an alias  
{{% /alert %}}
```

The support team member's official GitLab email.

title

The support team member's job title

entity

The GitLab entity the team member belongs to.

A list of available entities is:

- GitLab BV
- GitLab Canada Corp
- GitLab GmbH
- GitLab Inc
- GitLab IT BV
- GitLab Korea LTD
- GitLab PTY Ltd
- GitLab PTY Ltd NZ
- GitLab Singapore PTE. LTD
- GitLab UK Ltd

workinghours

A String representing your working hours. You should specify the timezone to be safe. (examples being 0800-1600 Central time or 0000-0800 UTC).

reportsto

The name of the person to whom the support team member reports

region

The support team member's region

The current regions we have available are:

- AMER-W
- AMER-C
- AMER-E
- APAC
- EMEA

location

The city/state in which the support team member resides

tzdatatimezone

The support team member's timezone

country

The country in which the support team member resides

startdate

The date the support team member started at GitLab

calendly

The link to the support-team member's calendly. Should always be the link to their main page, not an event.

languages

An array of languages the support team member knows

pagerduty

A hash containing Pagerduty information

- Should consist of an attribute id, which is the support team member's Pagerduty user ID

focuses

An array containing the support team member's focuses

Each focuses array contains a hash with the attributes of:

- name, which is the name of the focus
- percentage, which is the percentage of the focus

All percentages should add up to be 100.

A list of current focuses available are:

- Onboarding - focused on onboarding to their position
- SaaS - focused on working Global SaaS support tickets
- License and Renewals - focused on working Global L&R support tickets
- Self-Managed - focused on working Global Self-Managed support tickets
- Operations - focused on Support Operations responsibilities
- Management - focused on Support Leadership responsibilities
- US Federal - focused on working US Federal support tickets
- ASE - focused on Assigned Support Engineer work

Additional info about the ASE focus

When the name of the focus is ASE, additional attributes can be added to the hash:

- zendesk, which specifies the instance the organizations are on
- organizations, which is an array of information about the specific organizations, having the attributes of:
 - id, which is the Zendesk ID for the organization
 - percentage, which specifies the percent of ASE time spent on that organization

As an example, if Bob spent 50% of his time as an ASE in the Global Zendesk instance, and managed two organizations (123 and 456) evenly, he would enter his ASE focus object like so:

```
yaml
focuses:
- name: 'Self-managed'
  percentage: 50
- name: 'ASE'
  percentage: 50
  zendesk: global
  organizations:
  - id: 123
    percentage: 50
  - id: 456
    percentage: 50
```

This means Bob spends 50% of his time doing Self-Managed work, and 50% of his time doing ASE work. And of that ASE work, 50% of it is for org 123 and 50% is for org 456.

Another example, if Bob spent 60% of his time as an ASE in the US Government, and managed two organizations (123 and 456) with different percentages, he would enter his ASE focus object

like so:

yaml

focuses:

- name: 'Self-managed'
percentage: 40
- name: 'ASE'
percentage: 60
zendesk: us-federal
organizations:
 - id: 123
percentage: 25
 - id: 456
percentage: 75

This means Bob spends 40% of his time doing Self-Managed work, and 60% of his time doing ASE work. And of that ASE work, 25% of it is for org 123 and 75% is for org 456.

zendesk

A hash containing Zendesk information.

The attributes are:

- main, which is a hash containing the information pertaining to Zendesk Global
- us-federal, which is a hash containing the information pertaining to Zendesk US Government

Each of those attributes have further child attributes within them:

- id, which is the support team member's Zendesk user ID
- groups, which is the groups the support team member's Zendesk user is in
- role, which is the role the support team member's Zendesk user has
- showinsignature, which is a hash containing an attribute of:
 - gitlabhandle, which tells the agent sync to add a line to your signature including your GitLab.com handle
- alias, which is what display name to use in Zendesk. If left blank, your full name will be used as the default
- salutations, which is an array of salutations to use in a support team member's Zendesk user's signature
- articlepublisher, which determines if you will have Zendesk article publisher rights or not

Additional info about the US Government round robin

For the us-federal attribute, there is an additional attribute:

- roundrobin, which is a Hash containing the attributes:
 - active:, which dictates whether you are part of the round robin or not
 - modifier, which is the modifier to multiply by when calculating your total workload weight. In the event this number is 0, the round robin will use 1

gitlab

A hash containing GitLab.com information.

The attributes are:

- id, which is the support team member's GitLab.com user ID
- username, which is the support team member's GitLab.com username
- worksaccount, which dictates whether the support team member works Account Deletion issues
- noteslink, which is a link the team member uses for notes

slack

A hash containing Slack information.

The attributes are:

- id, which is the support team member's Slack user ID
- username, which is the support team member's Slack handle

modules

An array of learning modules the support team member has completed

knowledgeareas

An array of areas in which the support team member is knowledgeable.

Each item is a Hash containing information about the support team member's perceived knowledgeable on along with the corresponding skill level they have on it.

Each item's attributes are:

- name, which is the knowledge area's name
- level, which is the support team member's knowledge level of the area
 1. Learning - I'm still learning the basics and I'm not yet ready to take tickets.
 1. Ready to work tickets - I'm ready to work on tickets, or learn by working on tickets.
 1. Looking to help others - I want to help others with tickets or learning.

hobbies

An array of the support team member's hobbies

pronouns

The preferred pronouns of the support team member

SECTION: security/customer-support-operations/docs/incident_io/_index.md

{{% pageinfo color="warning" %}}

This is an information page for the incident.io components Customer Support Operations utilizes.

If you are looking for information about managing them, please see workflows.

If you are looking for information on how it is used in incidents, please see Handling Incidents.

{{% /pageinfo %}}

Working with incidents

Creating an incident

To create an incident:

1. Navigate to Status pages
1. Click on the status page you want to make an incident on
1. Click Publish incident at the top-right
1. Fill out a meaningful Name
1. Set the Status of the incident
 - Investigating: Report an incident
 - Identified: Problem has been determined and a fix is being made
 - Monitoring: Fix is implemented and we are monitoring the situation
 - Resolved: Everything is good to go
1. Set a meaningful Message for the incident
1. Set the level of impact on Affected components
 - No impact: The incident does not impact this component
 - Degraded performance: The component is working but at lower than standard performance levels
 - Partial outage: Significant parts of the component are not working
 - Full outage: The component is hard down
1. Click Review incident
1. Review all information for accuracy
1. Click Publish incident
1. Publishing an update to an incident (including resolving it)

Updating an incident

To update an incident (including resolving it):

1. Navigate to Status pages
1. Click on the status page the incident is on
1. Click on the incident in question
1. Click the top-right status bar (says what the current status is)
1. Select the new Status

1. Enter a meaningful message
1. Click Review update
1. Review all information for accuracy
1. Click Publish update

Deleting an incident

To delete an incident:

1. Navigate to Status pages
1. Click on the status page the incident is on
1. Click on the incident in question
1. Click the ... at the top-right of the page
1. Click Delete status page incident
1. Click Delete incident

Working with maintenance

Scheduling maintenance

To create (schedule) maintenance:

1. Navigate to Status pages
1. Click on the status page you want to make an incident on
1. Click on Maintenance
1. Click Schedule maintenance at the top-right of the page
1. Fill out a meaningful Name
1. Ensure Automatically update status is checked
1. Set the Impact window (keep in mind it is using local time)
1. Set a meaningful Message
1. Set the level of impact on Affected components
 - No impact: The maintenance does not impact this component
 - Under maintenance: The maintenance does impact this component
1. Click Review
1. Review all information for accuracy
1. Click Publish maintenance

Updating maintenance

To update maintenance:

1. Navigate to Status pages
1. Click on the status page you want to make an incident on
1. Click on Maintenance
1. Click the maintenance in question
1. Click the top-right status bar (says what the current status is)
1. Select the new Status
1. Enter a meaningful message
1. Click Review update
1. Review all information for accuracy

1. Click Publish update

Deleting maintenance

To delete maintenance:

1. Navigate to Status pages
1. Click on the status page you want to make an incident on
1. Click on Maintenance
1. Click the maintenance in question
1. Click the ... at the top-right of the page
1. Click Delete status page incident
1. Click Delete incident

Our Setup

Components catalog

Source: Customer Support Operations components

This contains all the pieces we would report outages on. The steps to create it were as follows:

1. Go to Catalog
1. Click Add a custom type
1. Enter a Name
1. Enter a Description
1. Under Categories click + Services
1. Ensure Are entries of this type ranked? checkbox is checked
1. Ensure Reference entries by name checkbox is checked
1. Click Save at bottom-right

Once created, components are added via the following steps:

1. Go to Catalog
1. Click the catalog to add components to
1. Click Create entry at the top-right
1. Enter a Name
1. Click Create at the bottom-right
1. Repeat as needed

Groups catalog

Source: Customer Support Operations groups

This contains all the groups we would use (we only use one currently). The steps to create it were as follows:

1. Go to Catalog
1. Click Add a custom type

1. Enter a Name
1. Enter a Description
1. Under Categories click + Services
1. Ensure Are entries of this type ranked? checkbox is checked
1. Ensure Reference entries by name checkbox is checked
1. Click Save at bottom-right

Once created, groups are added via the following steps:

1. Go to Catalog
1. Click the catalog to add groups to
1. Click Create entry at the top-right
1. Enter a Name
1. Select all components applicable
1. Enter gitlab.com for the Email domains
1. Click Create at the bottom-right
1. Repeat as needed

Status page

Live at <https://statuspage.incident.io/cust-support-ops/main> (requires use of gitlab.com email address for authentication)

Source: Customer Support Operations Status

.

This is the actual status page. The steps to create it were as follows:

1. Go to Status pages
1. Click Create customer page
1. Enter a Page title
1. Enter Status page URL (if you want to change the auto-generated one)
1. Under Customer pages
 - Click Use an existing catalog type
 - Set Which catalog type defines the customers you'd like to create? to the Groups catalog created previously
 - Set Which catalog attribute represents your components? to Components
 - Set Which catalog attribute represents allowed email domains? to Email domains
1. Under Pages
 - Set the Page name
 - Set the URL
1. Under Page setup
 - Set Theming to Dark mode
 - Select a Company logo (if desired)
 - Select a Favicon (if desired)
1. Click Create status page at the bottom-right of the page

SECTION: security/customer-support-operations/docs/pagerduty/_index.md

{{% pageinfo color="warning" %}}

This is an information page for the Pagerduty components Customer Support Operations utilizes.

If you are looking for information about using or managing them, please see workflows.

{{% /pageinfo %}}

Schedules

We currently utilize the following schedules:

- Customer Support Operations
 - Timezone: UTC
 - Layers:
 - AMER
 - Rotation type: weekly
 - Handoff time: Monday 04:00 PM
 - Restrict on-call shifts to specific times:
 - from 04:00 PM to 12:00 AM
 - APAC 1
 - Rotation type: weekly
 - Handoff time: Monday 12:00 AM
 - Restrict on-call shifts to specific times:
 - from 12:00 AM to 4:00 AM
 - APAC 2
 - Rotation type: weekly
 - Handoff time: Monday 04:00 AM
 - Restrict on-call shifts to specific times:
 - from 04:00 AM to 08:00 AM
 - EMEA
 - Rotation type: weekly
 - Handoff time: Monday 08:00 AM
 - Restrict on-call shifts to specific times:
 - from 08:00 AM to 04:00 PM

Escalation Policies

We currently utilize the following escalation policies:

- Customer Support Operations
 - Immediately after an incident is triggered
 - Notify the following users or schedules
 - Customer Support Operations
 - escalates after 10 minutes
 - Notify the following users or schedules
 - Jason Colyer
 - escalates after 10 minutes
 - Notify the following users or schedules
 - Lyle Kozloff

- escalates after 10 minutes
- If no one acknowledges, repeat this policy 5 times

Services

We currently utilize the following services:

- Customer Support Operations
 - Integrations
 - None
 - Workflows
 - None
 - Settings
 - Assign and Notify
 - Assign to escalation policy: Customer Support Operations
 - How should responders be notified: High-urgency notifications, escalate as needed
 - When incidents are not actioned, automatically:
 - All options unchecked
 - Reduce Noise
 - Currently not grouping alerts on this service.
 - Coordinate Responders and Stakeholders
 - Conference Bridge Dial-In Number: none
 - Conference Bridge Meeting URL: none
 - Event Management
 - Not used
 - Remediate
 - Documentation link: No documentation link listed for this service
 - Custom Incident Actions: No custom incident actions on this service
 - Service Dependencies
 - None

SECTION: security/customer-support-operations/docs/ruby-gems/_index.md

SECTION: security/customer-support-operations/docs/ruby-gems/gitlab-support-readiness-gem.md

^{Introduced via [gitlab-com/support/support-ops/support-ops-project1919](https://gitlab.com/support/support-ops/support-ops-project1919)}

{{% pageinfo color="warning" %}}

This is an information page for the gem. If you are looking for information about maintaining the gem, please see Ruby gems workflows

{{% /pageinfo %}}

The
gitlabsupportreadiness gem
is our own created gem to act as our core library for all that we do in Support
Readiness.

Administration information

How do I install it on my system?

You can install a gem using the gem install command. To install our specific
gem, you would run:

```
text  
gem install gitlabsupportreadiness
```

How do I update it on my system?

You can update the gem on your system by running the command

```
text  
gem update gitlabsupportreadiness
```

How do I include it in my code?

Much like any other gem, you simply require it in your code and it will be used.
You can do this using

```
ruby  
require 'supportreadiness'
```

Usage

More detailed information on how to use the gem can be found within the
YARD docs
for the gem itself. To keep our handbook clean, we won't go over every
applicable use of the gem, but instead show some general examples.

Example: Adding overrides to a Pagerduty schedule

To do this we need to define the configuration to use for the Pagerduty
connection, establish the client, locate our schedule, and then create
overrides. Overall, this would look like:

```
ruby  
require 'supportreadiness'  
  
config = Readiness::Pagerduty::Configuration.new
```

```

config.token = ENV.fetch('PDWRITETOKEN')
client = Readiness::Pagerduty::Client.new(config)
schedule = Readiness::Pagerduty::Schedules.find(client, 'ABC123')
overrides = [
  {
    start: '2012-07-01T00:00:00-04:00',
    end: '2012-07-02T00:00:00-04:00',
    user: {
      id: 'PEYSGVA',
      type: 'userreference'
    },
    timezone: 'UTC'
  }
]
created = Readiness::Pagerduty::Schedules.createoverrides(client, schedule, overrides)
pp created.map { |c| c['status'] }
=> [201]

```

A breakdown of what this is doing would be:

- config = Readiness::Pagerduty::Configuration.new

Setting the variable config to a new instance of the Readiness::Pagerduty::Configuration Object

- config.token = ENV.fetch('PDWRITETOKEN')

Setting the instance variable token in the Object stored in the variable config to the value stored within the environment variable PDWRITETOKEN

- client = Readiness::Pagerduty::Client.new(config)

Set the variable client to a new instance of the Readiness::GitLab::Client Object, passing the value of the variable config as a parameter.

- schedule = Readiness::Pagerduty::Schedules.find(client, 'ABC123')

Set the variable schedule to the value returned by running the function Readiness::Pagerduty::Schedules.find!, passing the variable client and the String ABC123 as parameters. This function will use the Pagerduty API to find a schedule and convert its information into an Object

- overrides = [..]

Set the variable overrides to the an Array (see Pagerduty docs for more details on what to use for overrides)

- created = Readiness::Pagerduty::Schedules.createoverrides(client, schedule, overrides)

Set the variable created to the value returned by running the function `Readiness::Pagerduty::Schedules.createoverrides`, passing the variables `client`, `schedule`, and `overrides` as parameters. This function will use the Pagerduty API to create overrides in that schedule

```
- pp created.map { |c| c['status'] }
```

Outputs the value of the key `status` from within the Array of Hashes stored in the variable created

Example: Fetching the value stored within a key from Redis

To do this, we need to establish the client for the Redis connection and then fetch the value from the key. Overall, this would look like:

```
ruby
require 'supportreadiness'

redis = Readiness::Redis.new(
  ENV.fetch('REDISHOSTNAME'),
  ENV.fetch('REDISPORT'),
  ENV.fetch('REDISPASSWORD')
)
pp Readiness::Redis.get(redis, 'foo')
=> "bar"
```

A breakdown of what this is doing would be:

```
- redis = Readiness::Redis.new(
```

Set the variable `redis` to a new instance of the `Readiness::Redis` Object, passing several parameters (see below)

```
- ENV.fetch('REDISHOSTNAME')
```

Use the value stored within the environment variable `REDISHOSTNAME`

```
- ENV.fetch('REDISPORT')
```

Use the value stored within the environment variable `REDISPORT`

```
- ENV.fetch('REDISPASSWORD')
```

Use the value stored within the environment variable `REDISPASSWORD`

```
- pp Readiness::Redis.get(redis, 'foo')
```

Outputs the value stored within the key `foo`.

Example: Adding a comment and updating the status of a ticket in Zendesk Global

To do this, we need to define the configuration to use for the Zendesk connection, establish the client, locate our ticket, and then update its status. Overall, this would look like:

```
ruby
require 'supportreadiness'

config = Readiness::Zendesk::Configuration.new
config.username = ENV.fetch('ZENDESKGLOBALUSERNAME')
config.token = ENV.fetch('ZENDESKGLOBALTOKEN')
config.url = ENV.fetch('ZENDESKGLOBALAPIURL')
client = Readiness::Zendesk::Client.new(config)
ticket = Readiness::Zendesk::Tickets.find!(client, 123456)
ticket.status = 'open'
ticket.comment = { body: 'I have an update!', authorid: 123456789 }
update = Readiness::Zendesk::Tickets.update!(client, ticket)
pp update.status
=> "open"
```

A breakdown of what this is doing would be:

- config = Readiness::Zendesk::Configuration.new

Setting the variable config to a new instance of the Readiness::Zendesk::Configuration Object

- config.username = ENV.fetch('ZENDESKGLOBALUSERNAME')

Setting the instance variable token in the Object stored in the variable config to the value stored within the environment variable ZENDESKGLOBALUSERNAME

- config.token = ENV.fetch('ZENDESKGLOBALTOKEN')

Setting the instance variable token in the Object stored in the variable config to the value stored within the environment variable ZENDESKGLOBALTOKEN

- config.url = ENV.fetch('ZENDESKGLOBALAPIURL')

Setting the instance variable token in the Object stored in the variable config to the value stored within the environment variable ZENDESKGLOBALAPIURL

- client = Readiness::Zendesk::Client.new(config)

Set the variable client to a new instance of the

Readiness::Zendesk::Client Object, passing the value of the variable config as a parameter.

```
- ticket = Readiness::Zendesk::Tickets.find!(client, 123456)
```

Set the variable ticket to the value returned by running the function Readiness::Zendesk::Tickets.find!, passing the variable client and the Integer 123456 as parameters. This function will use the Zendesk API to find a ticket and convert its information into an Object

```
- ticket.status = 'open'
```

Setting the instance variable status in the Object stored in the variable ticket to a String

```
- ticket.comment = { body: 'I have an update!', authorid: 123456789 }
```

Setting the instance variable status in the Object stored in the variable ticket to a Hash

```
- update = Readiness::Zendesk::Tickets.update!(client, ticket)
```

Set the variable update to the value returned by running the function Readiness::Zendesk::tickets.update!, passing the variables client and ticket as parameters. This function will use the Zendesk API to update the ticket and convert its information into an Object

```
- pp update.status
```

Outputs the value of the instance variable status on the Object stored within the variable update

Example: Creating a gitlab.com issue

To do this, we need to define the configuration to use for the gitlab.com connection, establish the client, locate our project, and then create an issue. Overall, this would look like:

```
ruby
require 'supportreadiness'
```

```
config = Readiness::GitLab::Configuration.new
config.token = ENV.fetch('GITLABPAT')
client = Readiness::GitLab::Client.new(config)
project = Readiness::GitLab::Projects.find!(client, 123)
issue = Readiness::GitLab::Issues.new
issue.title = 'Important issue!'
issue.description = 'This is totally important!'
issue.assigneeids = [1, 3, 4]
createdissue = Readiness::GitLab::Issues.create!(client, project, issue)
```

```
puts createdissue.weburl  
=> "https://gitlab.com/jcolyer/myproject/-/issues/8"
```

A breakdown of what this is doing would be:

```
- config = Readiness::GitLab::Configuration.new
```

Setting the variable config to a new instance of the Readiness::GitLab::Configuration Object

```
- config.token = ENV.fetch('GITLABPAT')
```

Setting the instance variable token in the Object stored in the variable config to the value stored within the environment variable GITLABPAT

```
- client = Readiness::GitLab::Client.new(config)
```

Set the variable client to a new instance of the Readiness::GitLab::Client Object, passing the value of the variable config as a parameter.

```
- project = Readiness::GitLab::Projects.find!(client, 123)
```

Set the variable project to the value returned by running the function Readiness::GitLab::Projects.find!, passing the variable client and the Integer 123 as parameters. This function will use the gitlab.com API to find a project and convert its information into an Object

```
- issue = Readiness::GitLab::Issues.new
```

Set the variable issue to a new instance of the Readiness::GitLab::Issues Object

```
- issue.title = 'Important issue!'
```

Setting the instance variable title in the Object stored in the variable issue to a String

```
- issue.description = 'This is totally important!'
```

Setting the instance variable description in the Object stored in the variable issue to a String

```
- issue.assigneeids = [1, 3, 4]
```

Setting the instance variable assigneeids in the Object stored in the variable issue to an Array

```
- createdissue = Readiness::GitLab::Issues.create!(client, project, issue)
```

Set the variable `createdissue` to the value returned by running the function `Readiness::GitLab::Issues.create!`, passing the variables `client`, `project`, and `issue` as parameters. This function will use the `gitlab.com` API to create an issue and convert its information into an Object

- puts `createdissue.weburl`

Outputs the value of the instance variable `weburl` on the Object stored within the variable `createdissue`

Useful links

- Source code
- rubygems.org page
- Issue tracker
- YARD docs
- CHANGELOG

SECTION: security/customer-support-operations/docs/salesforce/_index.md

SECTION: security/customer-support-operations/docs/salesforce/skus.md

{{% pageinfo color="info" %}}

This is an informational page for the SKU mapping.

If you are looking for information about maintaining it, please see our workflows

{{% /pageinfo %}}

While we call them "SKUs", in the actual code what we are mapping is Product Rate Charges.

Due to the sheer number of them, we document the mapping via the Product Rate Charge Name mapping google sheet (internal only).

SECTION: security/customer-support-operations/docs/slack/_index.md

SECTION: security/customer-support-operations/docs/slack/notify-oncall.md

^{Introduced via [gitlab-com/support/support-team-meta6109](https://gitlab.com/support/support-team-meta6109)}

{{% pageinfo color="info" %}}

This is an informational page for the notify oncall mechanism. It may not reflect the way we actually manage it.

If you are looking for information about maintaining it, please see our workflows

{{% /pageinfo %}}

Where does it live

It lives within the Readiness namespace at the notify-oncall project.

How does it work

A Pagerduty webhook is used when an incident is triggered on the Customer Support - US Federal service, which in turn triggers a CI/CD pipeline within the notify-oncall project.

This then processes the payload from the Pagerduty webhook to determine if a human is currently assigned to be oncall. If one is, the scripts exit with a 0 status code.

If no human is currently assigned, the scripts then determine a list of US Government first shift agents who are available (i.e. not on PTO). Using this list, the scripts ping the available agents via Slack in the sptus-government Slack channel.

SECTION: security/customer-support-operations/docs/slack/vbt-slackbot.md

^{Introduced via [gitlab-com/support/support-team-meta5486](https://gitlab.com/support/support-team-meta5486)}

{{% pageinfo color="info" %}}

This is an informational page for the VBT Slackbot. It may not reflect the way we actually manage it.

If you are looking for information about maintaining it, please see our workflows

{{% /pageinfo %}}

Where does it live

It lives within the Readiness namespace at the Very Breached Ticket Slackbot project.

How does it work

It runs via the following scheduled pipelines:

Name	Interval
Managers AMER	0 14 1-5 Etc/UTC
Managers APAC	30 21 0-4 Etc/UTC
Managers EMEA	0 7 1-5 Etc/UTC

During each run, a Zendesk view preview is generated using the following criteria:

- ALL
 - Time since SLA breach is greater than 48 hours
 - Ticket form is not Billing
 - Ticket form is not Support Ops
 - Ticket status is less than solved (i.e. new, open, pending, on-hold)
- ANY
 - Ticket stage is Emergencies
 - Ticket stage is FRT

The tickets showing in the Zendesk view preview using those conditions are then used in generating a message that is posted to via Slack in the sptleaders-daily channel.

SECTION: security/customer-support-operations/docs/slack/workflows.md

{{% pageinfo color="info" %}}

This is an informational page for Slack workflows. It may not reflect the way we actually manage Slack workflows.

If you are looking for information about maintaining Slack workflows (creating, editing, etc.), please see our workflows

{{% /pageinfo %}}

Building a Slack workflow

To build a brand new workflow, you will first click the 3 horizontal dots on the left-hand side of your Slack window (the More option) and select Automations.

From this page, you will want to click Workflow builder button on the left-hand side of the page, which should make a new window pop-up.

Within the new window, click the green Create Workflow button to start the process.

From here, you will select the various steps you want to occur. Using the pre-made templates is often the best place to start, but feel free to start from

scratch if you feel comfortable doing so.

Once you are done building all the steps you want in the workflow, you will click the green Finish Up button at the top-right of the window. Doing so will normally prompt you to add the Name, Description, Image, and Workflow managers.

The exact options you pick depend on the nature of your workflow, but in terms of workflows managed by Customer Support Operations, we use the following options (you might need to click Show more permissions to see all the options):

- Workflow managers:
 - Jason Colyer
 - Nabeel Hasan
 - Alyssa Villa - Dionela
 - Dylan Tragjasi
 - Sarah Cole
 - Rene Verschoor
 - Lyle Kozloff
- Can find this workflow
 - Everyone in GitLab
- Can copy this workflow
 - Workflow managers only

With all that in place, click the green Publish button to activate your workflow.

Remember to test it all out and update this page to reflect the new workflow (if it is Customer Support Operations managed)!

Modifying a Slack workflow

To edit an existing workflow, you will first click the 3 horizontal dots on the left-hand side of your Slack window (the More option) and select Automations.

From this page, you will want to click Managed by you button on the left-hand side of the page and then select the workflow you are wanting to modify (and then click the Edit button), which should make a new window pop-up.

From here, you are in the workflow builder. Make the changes you want to make to the workflow and then click the green Publish button at the top-right of the window.

Remember to test it all out and update this page to reflect the new workflow (if it is Customer Support Operations managed)!

Unpublishing and deleting a Slack workflow

To unpublish or delete a workflow, you will follow the same directions as the Modifying a Slack workflow section. Instead of

modifying steps and the like though, you are going to click the 3 horizontal dots at the top-right of the page, then select the option you are looking for (either Unpublish or Delete).

Remember to test it all out and update this page to reflect the new workflow (if it is Customer Support Operations managed)!

Currently used Slack workflows

This section serves as a form of "backup" for the Slack workflows we manage, as the current implementation of Slack workflows does not allow an export/import option.

Support Contact Management

Locate at
Support Contact Management

1. Choose how to start the workflow

- When an emoji reaction is used
- Emoji reaction:
 - :admissiontickets:
- Channels:
 - supportgitlab-com
 - supportleadership
 - supportlicensing-subscription
 - supportoperations
 - supportself-managed
 - supportticket-attention-requests

1. Reply to a message in thread

- Select a message to reply to: Reference to the message that was reacted to
- Add a reply:
 - > Hello VARIABLE!
 - >
 - > The management of support contacts for a Zendesk Global organization can
 - > only be done by the users of the organization, and will be done via a
 - > Support Operations ticket. The information for this can be found at
 - > <https://about.gitlab.com/support/managing-support-contacts/>. This page
 - > also contains the exact links to use to contact the Support Ops team in
 - > regards to contact management. The specific form linked on this page will
 - > not automatically close out a case.
 - >
 - > Note that nothing that is auto-generated on the page, such as the
 - > Choose the reason why you are reaching out to us today, should be
 - > changed to ensure proper ticket routing.
 - >
 - > Please direct your customer to that page and encourage them to fully read
 - > the information therein. From there, they should make a decision on the
 - > next course of action in regards to their support contacts that best
 - > suits their company.

- Variables
- User who sent the message that was reacted to (@display name)

Permissions:

- Managers:
 - Jason Colyer
 - Nabeel Hasan
 - Alyssa Villa - Dionela
 - Dylan Tragjasi
 - Sarah Cole
 - Rene Verschoor
 - Lyle Kozloff
- Can find this workflow
 - Everyone in GitLab
- Can copy this workflow
 - Workflow managers only

SECTION: security/customer-support-operations/docs/supportlogic/_index.md

Description

SupportLogic is a customer support AI platform that uses natural language processing to analyze customer interactions and predict customer escalations before they happen.

Getting access

Access is provisioned through Lumos.

SECTION: security/customer-support-operations/docs/transcend/_index.md

Description

Transcend is a tool used for servicing data privacy requests. The primary users are Support Engineers and the Privacy team. System ownership may at some point be transferred to another team, but for now it's maintained by Customer Support Operations.

Access

Access is through Lumos. Please submit a request there.

SECTION: security/customer-support-operations/docs/workato/_index.md

{{% pageinfo color="info" %}}

This is an informational page for Workato. It may not reflect the way we actually manage it.

If you are looking for information about maintaining it, please see our workflows

{{% /pageinfo %}}

What is Workato

According to wikipedia:

> Workato is an American multinational software company that provides a cloud-based platform for automation, integration, and artificial intelligence (AI) orchestration across applications, data, and systems.

In a broader sense, Workato is a tool we utilize to enhance the functionality of many of our systems, such as Zendesk, GitLab.com, etc.

Creating a recipe

For information on creating a recipe, please see Workato's documentation

Creating a connection

For information on creating a connection, please see Workato's documentation

SECTION: security/customer-support-operations/docs/zendesk/_index.md

Instances

Zendesk Global

This is our main (and first) Zendesk instance. Most people interacting with GitLab Support will use this instance.

- Accessible by: Everyone
- Support portal address: <<https://support.gitlab.com>>
- Sandbox address: <<https://gitlab1707170878.zendesk.com/agent/>>

Zendesk US Government

This is a Zendesk instance made specifically with Self-Managed public sector users in mind. It has many restrictions and is not generally usable without pre-approval.

- Accessible by: US Citizens only
- Support portal address: <<https://gitlab-federal-support.zendesk.com>>
- Sandbox address: <<https://gitlabfederalsupport1585318082.zendesk.com/agent>>

SECTION: security/customer-support-operations/docs/zendesk/agents.md

{{% alert title="Note" color="primary" %}}

This is an informational page for the Zendesk agents. It may not reflect the way we actually manage Zendesk agents.

- If you are looking for information about modifying the agent sync, please see agent sync workflow
- If you are looking for information about provisioning agents, please see provisioning workflow

{{% /alert %}}

In Zendesk, agents are those who have abilities beyond end-users (and thus access the agent workspace).

We largely manage agents via the agent sync (for the Support team) and via provisioning/deprovisioning.

Agent Sync

{{% alert title="Note" color="primary" %}}

This solely applies to those who have YAML files in Support team YAML files

{{% /alert %}}

The agent sync is how we manage the Support teams various metadata within Zendesk. This is run on a scheduled pipeline via the corresponding project:

- Zendesk Global
- Zendesk Us Federal

Each run will update the following for the agents:

- GitLab.com user ID
- GitLab.com username
- Groups
- Manager tag
- Name
- Out of Office status
- Signature
- Some user tags
- User region (Zendesk Global only)

Any modifications to a Support agent in Zendesk will be overridden by this.

Manually creating an agent

```
{{% alert title="Note" color="danger" %}}
```

This should never be done unless specifically indicated in workflow, such as the provisioning workflow

```
{{% /alert %}}
```

To create an agent in Zendesk:

1. a

```
## SECTION: security/customer-support-operations/docs/zendesk/api.md
```

```
{{% alert title="Note" color="primary" %}}
```

This is an informational page for the Zendesk api. It may not reflect the way we actually manage Zendesk api (and its tokens).

If you are looking for information about maintaining api tokens (creating, editing, etc.), please see API workflow

```
{{% /alert %}}
```

What is the Zendesk API

The Zendesk Support API is a collection of Zendesk endpoints you can use to get various information or do various tasks. It is quite robust and something we use from time to time.

What are Zendesk API tokens

Zendesk API tokens are used in authentication for Zendesk API requests. These tokens are always at the administrator level and cannot be issued at lower permission/role levels. As such, you should always use caution in using and issuing these tokens.

How to authenticate to the Zendesk API

Basic authentication

To authenticate using basic authentication, you'll need to know your username (email) and password for your Zendesk account. With those in hand, you can either use those directly or encode the string into base64 (and use it in the headers).

Example of using it raw:

bash

```
curl https://gitlab.zendesk.com/api/v2/users.json \
-u jcolyer@gitlab.com:mypassword
```

Example of using it via headers

```
bash
echo 'jcolyer@gitlab.com:mypassword' | base64
amNvbHllckBnaXR5YWluY29tOm15X3Bhc3N3b3JkCg==

curl https://gitlab.zendesk.com/api/v2/users.json \
-H "Authorization: Basic amNvbHllckBnaXR5YWluY29tOm15X3Bhc3N3b3JkCg=="
```

API token authentication

To authenticate via an API token, you'll need to know your username and the API token in question. With those in hand, you can either use those directly or encode the string into base64 (and use it in the headers). When using an API token, you must add /token after your username.

```
bash
curl https://gitlab.zendesk.com/api/v2/users.json \
-u jcolyer@gitlab.com/token:apitoken
```

Example of using it via headers

```
bash
echo 'jcolyer@gitlab.com/token:apitoken' | base64
amNvbHllckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2t1bgo=

curl https://gitlab.zendesk.com/api/v2/users.json \
-H "Authorization: Basic amNvbHllckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2t1bgo="
```

If you've followed the recommended setup and stored your credentials in environment variables, the call will be simplified to

```
bash
curl -u $ZDUSERNAME/token:$ZDTOKEN "$ZDURL/users.json"
```

Oauth access token authentication

To use an Oauth access token, you'd first need to create an Oauth app in Zendesk (see Zendesk docs for more information). With the access token in hand, you would pass this into the headers.

Example of using it via headers

```
bash
curl https://gitlab.zendesk.com/api/v2/users.json \
  -H "Authorization: Bearer gErypPlm4dOVgGRvA1ZzMH5MQ3nLo8bo"
```

How to use the Zendesk API

```
{{% alert title="Note" color="primary" %}}
```

This focuses solely on the Zendesk API via curl. For more information on using a library, check out the corresponding library's documentation.

```
{{% /alert %}}
```

To get started, you would need to know the endpoint you wish to use. Generally speaking, the most common ones you might use for quick actions are:

- Zendesk Support API Tickets endpoints
- Zendesk Support API Users endpoints
- Zendesk Support API Organizations endpoints

For more administrative tasks, the common ones you might use are:

- Zendesk Support API Ticket Forms endpoints
- Zendesk Support API Ticket Fields endpoints
- Zendesk Support API Views endpoints
- Zendesk Support API Triggers endpoints
- Zendesk Support API Macros endpoints
- Zendesk Support API Automations endpoints

Once you have determined what you wish to do, go to the corresponding API endpoint documentation and make note of:

- The request type
 - GET
 - POST
 - PUT
 - DELETE
- The endpoint URL
- Any required parameters

From there, you will craft a curl command in the format of:

```
bash
curl ZENDESKURL/api/v2/ENDPOINT \
  -X REQUESTTYPE \
  -H HEADERINFO \
  -u AUTHENTICATION \
  -d DATATOUSE
```

Where:

- ZENDESKURL is the URL of the Zendesk instance
- ENDPOINT is the endpoint to use
- -X REQUESTTYPE is the request type from the documentation (not needed if making a GET request)
- -H HEADERINFO is any needed header information (not always needed)
- -u AUTHENTICATION is the user/pass or user/token combo (not needed if using header based authentication)
- -d DATETOUSE is the data to send with the request (not always needed)

As an example, if you wanted to get the details for automation 12345, your curl command might look like:

```
bash
curl https://gitlab.zendesk.com/api/v2/automations/12345 \
  -H "Authorization: Basic amNvbHllckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Likewise, if you wanted to update automation 12345 to set active to false (i.e. deactivate it), your curl command might look like:

```
bash
curl https://gitlab.zendesk.com/api/v2/automations/12345 \
  -H "Authorization: Basic amNvbHllckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo=" \
  -H "Content-Type: application/json" \
  -X PUT \
  -d '{"automation": {"active": false}}'
```

If we wanted to full on delete automation 12345, your curl command might look like:

```
bash
curl https://gitlab.zendesk.com/api/v2/automations/12345 \
  -H "Authorization: Basic amNvbHllckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo=" \
  -H "Content-Type: application/json" \
  -X DELETE
```

Common use cases

For all examples:

- 12345 refers to the items ID number

Automations

List all

bash

```
curl https://gitlab.zendesk.com/api/v2/automations \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Show details

bash

```
curl https://gitlab.zendesk.com/api/v2/automations/12345 \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Deactivate

bash

```
curl https://gitlab.zendesk.com/api/v2/automations/12345 \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo=" \
-H "Content-Type: application/json" \
-X PUT \
-d '{"automation": {"active": false}}'
```

Macros

List all

bash

```
curl https://gitlab.zendesk.com/api/v2/macros \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Show details

bash

```
curl https://gitlab.zendesk.com/api/v2/macros/12345 \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Deactivate

bash

```
curl https://gitlab.zendesk.com/api/v2/macros/12345 \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo=" \
-H "Content-Type: application/json" \
-X PUT \
-d '{"macro": {"active": false}}'
```

SLA Policies

List all

bash

```
curl https://gitlab.zendesk.com/api/v2/slas/policies \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Show details

bash

```
curl https://gitlab.zendesk.com/api/v2/slas/policies/12345 \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Tickets

List all

bash

```
curl https://gitlab.zendesk.com/api/v2/tickets \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Show details

bash

```
curl https://gitlab.zendesk.com/api/v2/tickets/12345 \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Change status to closed

bash

```
curl https://gitlab.zendesk.com/api/v2/tickets/12345 \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo=" \
-H "Content-Type: application/json" \
-X PUT \
-d '{"ticket": {"status": "closed"}}'
```

Triggers

List all

bash

```
curl https://gitlab.zendesk.com/api/v2/triggers \
-H "Authorization: Basic amNvbHlckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Show details

bash

```
curl https://gitlab.zendesk.com/api/v2/triggers/12345 \  
-H "Authorization: Basic amNvbHllckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Deactivate

bash

```
curl https://gitlab.zendesk.com/api/v2/triggers/12345 \  
-H "Authorization: Basic amNvbHllckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo=" \  
-H "Content-Type: application/json" \  
-X PUT \  
-d '{"trigger": {"active": false}}'
```

Views

List all

bash

```
curl https://gitlab.zendesk.com/api/v2/views \  
-H "Authorization: Basic amNvbHllckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Show details

bash

```
curl https://gitlab.zendesk.com/api/v2/views/12345 \  
-H "Authorization: Basic amNvbHllckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Deactivate

bash

```
curl https://gitlab.zendesk.com/api/v2/views/12345 \  
-H "Authorization: Basic amNvbHllckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo=" \  
-H "Content-Type: application/json" \  
-X PUT \  
-d '{"trigger": {"active": false}}'
```

List tickets in a view

bash

```
curl https://gitlab.zendesk.com/api/v2/views/12345/tickets \  
-H "Authorization: Basic amNvbHllckBnaXR5YWluY29tL3Rva2VuOmFwaV90b2tlbgo="
```

Count tickets in a view

```
bash
curl https://gitlab.zendesk.com/api/v2/views/12345/count \
-H "Authorization: Basic amNvbHlckBnaXRyY29tL3Rva2VuOmFwaV90b2t1bgo="
```

Creating an API token

For information on creating an API token, please see Zendesk documentation.

Deleting an API token

For information on deleting an API token, please see Zendesk documentation.

Useful links

- Zendesk API Docs
- Zendesk Oauth docs
- zendeskapiclient gem
- Zendesk Support API Tickets endpoints
- Zendesk Support API Users endpoints
- Zendesk Support API Organizations endpoints
- Zendesk Support API Ticket Forms endpoints
- Zendesk Support API Ticket Fields endpoints
- Zendesk Support API Views endpoints
- Zendesk Support API Triggers endpoints
- Zendesk Support API Macros endpoints
- Zendesk Support API Automations endpoints

SECTION: security/customer-support-operations/docs/zendesk/apps.md

{{% alert title="Note" color="primary" %}}

This is an informational page for the Zendesk apps. It may not reflect the way we actually manage Zendesk apps.

If you are looking for information about maintaining apps (creating, editing, etc.), please see Apps workflow

{{% /alert %}}

What is a Zendesk App

A Zendesk App is an application (written in HTML/CSS/JS) that runs in a location of Zendesk. What it does and how it does it varies greatly from application to application. Applications can be run in a great many places, but the traditional locations are:

- Ticket sidebar
- User sidebar

- Organization sidebar
- Navbar
- Background

You can see more resources on application locations via the Zendesk Developer Manifest Reference documentation.

Zendesk applications tend to come from one of two areas:

- Zendesk Marketplace
- Developed in-house

Installing a private app

<https://developer.zendesk.com/documentation/ticketing/using-the-zendesk-api/creating-and-managing-private-apps/creating-an-app>

Updating a private app

<https://developer.zendesk.com/documentation/ticketing/using-the-zendesk-api/creating-and-managing-private-apps/creating-an-app>

Zendesk Global apps

Advanced SAST App

^{Introduced via support-team-meta6652}

The Advanced SAST App is a ticket app that enables a quick working of User requests for source code of LGPL-licensed components in GitLab Advanced SAST.

App information:

- Located in the ticket sidebar
- Restricted by Group:
 - Support AMER
 - Support APAC
 - Support EMEA
- This application was developed in-house and can be found Advanced SAST App project.

Advanced Search

Advanced Search is an app that provides a simple visual interface for constructing complex search queries against tickets, users, and organizations (orgs). It also enables you to export the search results in a CSV format.

App information:

- Located in the navbar
- This application was developed by Zendesk and is available in the Zendesk Marketplace.

GitLab Reminders App

^{Introduced via support-team-meta3036}

The Reminders App appears in the navbar and allows the agent a more specialized view of tickets they are involved in. It currently shows:

- Tickets assigned to you with a pending/overdue task that are not in a Closed state
- Recent tickets you have viewed
- Tickets assigned to you that are not in a Closed state
- Tickets you are CC'd on that are not in a Closed state

It also allows you to quickly manage your tasks by seeing the notes you have left for said task, when it is due, and a button to quickly mark the task as done (remove the notes and due date).

App information:

- Located in the navbar
- This application was developed in-house and can be found GitLab Reminders App project.

GitLab Search

This app lets you use the gitlab.com API to search for issues/merge requests within Zendesk.

App information:

- Located in the topbar
- This application was developed in-house and can be found GitLab Search project.

GitLab Super App

^{Introduced via support-ops-project801}

A plugin controlled app that can do several things GitLab related

The current plugins are:

- User Lookup
 - > This lets you search gitlab.com for a username or email. It then displays information based on the results.
- Namespace Lookup
 - > This lets you search gitlab.com for a namespace. It then displays information based on the results.
- Collaboration Project
 - > This checks the organization for a collaboration project ID. If one exists, it then provides a link to said project.
- 2FA Helper
 - > This creates a usable form to checking if a 2FA verification has passed. It calculates the Risk Factor from the Data Classification and modifies it to reflect the passed challenges.

- Email Suppressions
 - > This searches mailgun for suppressions from bounces (note it does not do it on complaints or unsubscribes). It will display the results (with the message for the suppression).
 - >
 - > It also gives the option of removing the suppression (if one is found). Doing so deletes it from mailgun and adds an internal comment on the ticket with the results of the suppression deletion.
- Fieldnotes
 - > This app checks the Fieldnotes project for any existing Issues which reference the current Zendesk ticket ID. If no existing Issues are found, then agents are able to create a new Fieldnotes Issue from directly within the Zendesk ticket.
- 2FA Validator
 - > This app checks values entered by a support engineer and does a validation check. This effectively acts as the second layer of validation

App information:

- Located in the ticket sidebar
- Restricted by Group:
 - Support AMER
 - Support APAC
 - Support EMEA
- This application was developed in-house and can be found GitLab Super App project.

GitLab Views

The GitLab Views appears in the navbar and allows the agent a more customizable set of Zendesk views.

App information:

- Located in the navbar
- Restricted by Group:
 - Support AMER
 - Support APAC
 - Support EMEA
 - Support Managers
 - Support Ops
- This application was developed in-house and can be found GitLab Views App project.

Mechanizer

Please use CustomersDot Support Admin Tools instead of the app at this time.

^{Introduced via support-team-meta4208}

This app incorporates Mechanizer into Zendesk.

App information:

- Located in the ticket sidebar
- Restricted by Group:
 - Support AMER
 - Support APAC
 - Support EMEA
- This application was developed in-house and can be found at Mechanizer project

Notifications App

```
{{% alert title="Note" color="primary" %}}
```

This app is opt-in. This means nothing will happen unless you modify your user settings to receive notifications.

```
{{% /alert %}}
```

This app handles posting notifications at the top of the screen, depending on various conditions and user settings.

App information:

- Located in the Top Navigation of Zendesk Global
- Restricted by Group:
 - Support AMER
 - Support APAC
 - Support EMEA
 - Support Managers
 - Support Ops
- This app was developed in-house and can be found at Notification app project

Current events that trigger the app

The following events will send data to the app for notification processing:

- Agent private comment made on ticket
- Agent public comment made on ticket
- Customer public comment made on ticket
- Emergency ticket created
- Escalated ticket created
- Tickets being STAR'd
- Tickets created by specific organizations

User settings

The current user settings, which will determine what notifications you will (and will not) receive are:

- Play notification sound
 - Checking this box tells the app to play the notification sound.
- Notify me for

- This tells the app what kind of tickets to notify you for
- Values:
 - Assigned tickets only
 - CC'd tickets only
 - All tickets
- Notify me about
 - This tells the app what kind of events to notify you for
 - Values:
 - All public comments (agent and customer)
 - Only public comments from agents
 - Only public comments from customers
 - Only private comments
 - All types of comments
- Notify me only for tickets with priority
 - This tells the app which priorities to notify you on
 - Values:
 - at least Urgent
 - at least High
 - at least Medium
 - at least Low
 - Note A blank value is assumed to be "all priorities"
- Also notify me for escalated ticket creation
 - This dictates if you want to be notified via the app when an escalated organization creates a ticket.
 - Note This works independently of all other settings.
- Also notify me for emergency ticket creation
 - This dictates if you want to be notified via the app when an emergency ticket is created.
 - Note This works independently of all other settings.
- Also notify me for STARS
- Also notify me for soon to breach tickets on
 - This dictates if you want to be notified via the app when a ticket is about to breach (within 2 hours)
 - Values:
 - Assigned tickets only
 - CC'd tickets only
 - Tickets within my SGG only
 - All tickets
 - Note This works independently of all other settings.
- Also notify me for tickets created via specific orgs
 - This dictates if you want to be notified via the app when a ticket is created by a specific organization
 - The list should be comma separated
 - Example, if you want to be notified for organizations 123, 456, and 789, use the value 123,456,789 or 123, 456, 789
 - Note This works independently of all other settings.

For information on editing your personal user settings, please see Zendesk's documentation.

Out of Office

^{Introduced via support-team-meta4303}

This will enable an agent to mark when they are out of office in Zendesk, which then updates tickets and makes it visible in the views.

Managers are also able to do this for their reports.

App information:

- Located in the navbar
- Restricted by Group:
 - Support AMER
 - Support APAC
 - Support EMEA
- This application was developed in-house and can be found Out of Office project

Support Ops Super App

A plugin controlled app that can do several things Support Ops related

The current plugins are:

- Namespace Lookup
 - > This lets you search gitlab.com for a namespace. It then displays information based on the results. This is related to the one in the GitLab Super App, but instead it shows less information and shows the SFDC IDs it is associated with.
- Project Lookup
 - > This lets you search gitlab.com for a project. It then displays information based on the results.
- Attempt Association
 - > On tickets where the product type is GitLab.com, clicking the button on the plugin will attempt to auto-associate the requester to an organization. If that is not possible, it will detail why it was not possible.
- Associate User
 - > On a Support Ops ticket, it will ask you for an email address. It will then use the organization on the current ticket to associate said email address to that organization.
- CMP Developers
 - > Outputs a list of CMP developers (by email) for an organization (if it has a CMP)

App information:

- Located in the ticket sidebar
- Restricted by Role:
 - Admin
- This application was developed in-house and can be found Support Ops Super App project.

Unbabel

^{Introduced via support-team-meta1664}

Powered by state-of-the-art AI and a worldwide community of translators, Unbabel delivers translation at enterprise scale. We help you serve customers in any language, with fast, native-quality translations of your customer support tickets in Zendesk.

App information:

- Located in the ticket sidebar
- Restricted by Group:
 - Support AMER
 - Support APAC
 - Support EMEA
- This application was developed by Unbabel and is available in the Zendesk Marketplace.

Replying with a Translation

To request a translation automatically, simply reply as you normally would as an internal note with the unbabel hashtag included at the top of your content.

Please also ensure that the unbabelen, unbabelreply, unbabeled tags are included, otherwise your response might not be translated automatically. Should this happen, you will need to add the missing tags, and create a new internal note with the unbabel hashtag included at the top of your content.

Once you submit your response, it may take several seconds for Unbabel to automatically translate your internal comment, but it can take several minutes if a human is required to manually translate your internal comment. To view the status of the translation, you can open the Apps sidebar in the ticket, and scroll down to the Unbabel for Zendesk Support box.

After a translated response has been sent to the customer via Unbabel it is necessary to manually set the ticket status as Pending since Unbabel will incorrectly set the ticket status as Open. You must do this with an empty comment (remove any unbabel added by the plugin, before you Submit as Pending).

Excluding Text from Translation

The highlighted code can be skipped for translation by adding 3 brackets around the text:

```
string  
<<< text/code >>>
```

The above can also be used to protect sensitive information from a human translator when sending a translation request.

Disabling Unbabel in a Specific Ticket

Sometimes Unbabel is triggered if a customer's signature was written in a language that requires translation but the customer replies in English, and the translation is not needed. In this case, there is a way to disable Unbabel in this specific ticket:

- Open the ticket in question.
- Click Apps > Unbabel for Zendesk Support.
- Change the Customer language to English.

From now on, Unbabel will not be triggered in this ticket.

Help with Translation

If for some reason you have difficulty in understanding the automated translation, an actual human intervention can actually be requested. Simply click the link [Can't understand the translation?](#) in the Unbabel app box and this will send your response for translation to Unbabel editors.

Best Practices for Unbabel

As indicated in the training session, please keep in mind of the following best practices when writing a response for translation.

- Respond in one language
- It is likely that you can spe